

AFEC: Anchor-Frontier Enumeration with Cost-Split Compression for Subgraph Similarity Matching

Xinhao Zhang

Harbin Institute of Technology, Shenzhen

China

24s151181@stu.hit.edu.cn

Jianye Yang

Guangzhou University

China

jyyang@gzhu.edu.cn

Shengxin Liu

Harbin Institute of Technology, Shenzhen

China

sxliu@hit.edu.cn

Zhaoqun Gu

Harbin Institute of Technology, Shenzhen

China

guzhaoquan@hit.edu.cn

ABSTRACT

Subgraph similarity matching identifies approximate occurrences of a query graph in a large data graph. We study this problem under the missing-edge model, which allows a bounded number of query edges to be missing while requiring the preserved edges to connect all query vertices. Existing methods generate multiple relaxed query structures and apply exact matching to each structure. Overlaps among these structures can cause repeated search, duplicate results, and substantial intermediate-result overhead. We propose *Anchor-Frontier Enumeration* (AFE), a mapping-centric framework that directly enumerates vertex mappings and accounts for missing query edges incrementally. Its anchor-frontier mechanism branches on query edges between mapped and unmapped query vertices and partitions feasible mappings by the first such edge they preserve, enabling complete and duplicate-free enumeration. We further develop *Cost-Split*, which groups candidate data vertices that currently incur the same missing-edge cost and separates them only when later constraints distinguish them. Dynamic anchor-edge ordering and black-white selection further adapt the edge order and choose between individual and grouped candidate expansion. Together, these techniques form *Anchor-Frontier Enumeration with Cost-Split Compression* (AFEC), our complete enumeration algorithm. Experiments on 10 real-world graphs and 8,000 queries show that AFEC achieves $5.1\times$ – $159.9\times$ speedups over the fastest state-of-the-art baseline on each dataset and successfully processes 98.06% of the workload, compared with at most 69.46% for the baselines.

PVLDB Reference Format:

Xinhao Zhang, Shengxin Liu, Jianye Yang, and Zhaoqun Gu. AFEC: Anchor-Frontier Enumeration with Cost-Split Compression for Subgraph Similarity Matching. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/asdf1229/AFEC-SSM>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

1 INTRODUCTION

Graphs have become a fundamental model for representing complex relationships in many applications, such as biological networks, social networks, knowledge graphs, RDF data, and software analysis [7, 11, 23, 32, 43]. A core problem in graph analysis is subgraph matching [24, 28, 39]. Given a query graph Q and a large data graph G , exact subgraph matching enumerates all subgraph-isomorphic embeddings of Q in G , requiring every query edge to be preserved under the vertex mapping.

However, exact subgraph matching is often too restrictive in real-world applications. Real-world graph data are often noisy or incomplete, and query occurrences may differ by a small number of edges. Consequently, exact matching can miss structurally similar occurrences that remain meaningful. For example, in biological network analysis, a known pathway or interaction motif from one organism may be searched against the protein-protein interaction network of another organism, where some interactions may be absent because of biological variation or incomplete experimental observations [10, 26]. In software analysis, deviations from common object-interaction or API-usage patterns, such as missing method calls or interactions, may indicate anomalous or buggy code [29, 38]. In cyber threat hunting, attack behavior extracted from cyber threat intelligence can be represented as a query graph and matched against a provenance graph constructed from system audit logs, where incomplete detection or noisy observations may leave part of the expected attack structure observable [20, 31]. These applications motivate subgraph similarity matching, which relaxes strict edge preservation to identify approximate occurrences of a query graph.

In this paper, we study subgraph similarity matching under the missing-edge model on undirected vertex-labeled graphs [38, 41]. Given a query graph Q , a data graph G , and an integer threshold θ , a feasible match is an injective, label-preserving mapping that covers every query vertex, misses at most θ query edges, and keeps the preserved query graph connected. Exact subgraph matching is recovered when $\theta = 0$. We enumerate all feasible mappings.

Challenges and Existing Methods. The problem inherits the NP-hardness of exact subgraph matching [6], while allowing missing edges introduces additional combinatorial choices over which query edges are preserved. Existing methods generally solve this problem through a relaxation-based framework. They represent admissible missing-edge cases using multiple relaxed query structures, perform exact subgraph matching over these structures, and

subsequently assemble, verify, and deduplicate the resulting candidates [16, 36, 38, 41, 42]. *This design creates a mismatch between the search representation and the output representation: the search is organized around relaxed query structures, whereas each output is uniquely determined by a vertex mapping.* Because different relaxed structures may overlap or encode related missing-edge cases, they can repeatedly explore the same partial mappings and recover the same final mapping through multiple search paths. Methods that materialize large collections of intermediate matches may additionally incur substantial memory overhead.

Our Method. To eliminate structure-level redundancy, we propose *Anchor-Frontier Enumeration* (AFE), a mapping-centric framework that directly enumerates injective partial vertex mappings. Rather than constructing relaxed query structures, AFE determines preserved and missing query edges as a partial mapping grows and maintains the accumulated missing-edge cost. This aligns search with the output representation: a unified edit-aware enumeration replaces overlapping exact-matching tasks and avoids recovering the same mapping through different relaxations. The central challenge is preserving connectivity during direct mapping enumeration. Under the missing-edge model, a fixed connected query order does not guarantee that the next vertex has a preserved edge to the mapped portion; it may become connected only through unmapped vertices. Our key observation is that every feasible mapping must preserve at least one query edge between each unmapped query component and the mapped portion. AFE realizes this through an *anchor-frontier mechanism*: the unmapped endpoint of such an edge is a frontier vertex, its mapped endpoint serves as an anchor, and AFE branches on whether the corresponding anchor edge is preserved or missing. Sequential exclusions form an exclusion chain whose inclusion branches partition feasible completions by the first selected anchor edge they preserve. This partition enables complete and duplicate-free enumeration while retaining neighborhood-based candidate generation and exact missing-edge accounting.

AFE eliminates structure-level redundancy, but its inclusion branches still branch on every candidate of a frontier vertex. Before candidate choice affects constraints, these branches differ only in the mapping of the frontier vertex and otherwise repeat the same subsequent search. Existing aggregation techniques for exact matching cannot be applied directly because feasible candidates may incur different missing-edge costs and thus leave different remaining budgets. To reduce this candidate-level redundancy, we develop *Cost-Split*, a cost-aware candidate-set compression mechanism. Cost-Split aggregates only candidates with the same incremental missing-edge cost, charges their common cost immediately, and leaves the concrete mapping unresolved. When later edge or injectivity dependencies distinguish candidates by feasibility or incremental cost, Cost-Split refines only the affected candidate set. It thus replaces premature candidate-wise branching with cost-preserving compression while maintaining exact missing-edge accounting.

Finally, the effectiveness of AFE and Cost-Split depends on local branching decisions during enumeration. We therefore develop *dynamic anchor-edge ordering* to prioritize anchor edges expected to produce smaller inclusion branches and *dynamic black-white selection* to choose between black and white expansion based on their estimated branching scales. Together, AFE, Cost-Split, and

these adaptive strategies form *Anchor-Frontier Enumeration with Cost-Split Compression* (AFEC), our final enumeration algorithm.

Contributions. Our contributions are summarized as follows.

- We propose AFE, a mapping-centric, edit-aware framework that directly enumerates vertex mappings and incrementally accounts for missing query edges, avoiding the structure-level redundancy of relaxation-based methods. Its anchor-frontier mechanism maintains connectivity of the preserved query graph and partitions feasible mappings into disjoint branches, enabling complete and duplicate-free enumeration (Section 3).
- We develop *Cost-Split*, a cost-aware compression technique that reduces candidate-level redundancy by allowing candidates with the same incremental missing-edge cost to share exploration until constraints distinguish them (Section 4).
- We develop dynamic anchor-edge ordering and dynamic black-white selection to adapt the anchor order and expansion mode to each search state (Section 5).
- We conduct extensive experiments on 10 real-world data graphs and 8,000 queries. The results show that AFEC achieves $5.1\times$ – $159.9\times$ speedups over the fastest state-of-the-art baseline on each dataset and successfully processes 98.06% of the workload, compared with at most 69.46% for the baselines (Section 6).

In addition, Section 2 formalizes the problem and reviews existing relaxation-based methods, Section 7 discusses related work, and Section 8 concludes the paper.

2 PRELIMINARIES

2.1 Problem Definition

We focus on undirected and vertex-labeled simple graphs, i.e., graphs with neither self-loops nor parallel edges. Let Σ be a finite alphabet of vertex labels. For a graph H , we write $H = (V(H), E(H), L_H)$, where $V(H)$ is the vertex set, $E(H)$ is the edge set, and $L_H : V(H) \rightarrow \Sigma$ is the vertex-labeling function. For a vertex $x \in V(H)$, let $N_H(x) = \{y \in V(H) \mid (x, y) \in E(H)\}$ and $d_H(x) = |N_H(x)|$ denote its neighborhood and degree, respectively. For a vertex subset $U \subseteq V(H)$, $H[U]$ denotes the subgraph of H induced by U . Throughout the paper, we use $Q = (V(Q), E(Q), L_Q)$ and $G = (V(G), E(G), L_G)$ to denote the *query graph* and the *data graph*. For a partial mapping $M : U \rightarrow V(G)$ with $U \subseteq V(Q)$, we use $\text{dom}(M) = U$ and $\text{im}(M) = \{M(u) \mid u \in U\}$ to denote its domain and image, respectively. We assume that Q is connected.

We first review the definition of subgraph isomorphism.

Definition 1 (Subgraph Isomorphism [28]). Given a query graph Q and a data graph G , an injective mapping $f : V(Q) \rightarrow V(G)$ is a *subgraph isomorphism* from Q to G if it satisfies the two conditions:

- (1) *Label preservation*: $\forall u \in V(Q), L_Q(u) = L_G(f(u))$; and
- (2) *Edge preservation*: $\forall (u, u') \in E(Q), (f(u), f(u')) \in E(G)$.

Such a mapping f is also called an *exact match* of Q in G .

We next introduce the missing-edge model [38, 41], which relaxes edge preservation in subgraph isomorphism. Specifically, let $f : V(Q) \rightarrow V(G)$ be an injective, label-preserving mapping. The *preserved query graph* of Q under f is

$$Q_f = (V(Q), \{(u, u') \in E(Q) \mid (f(u), f(u')) \in E(G)\}, L_Q).$$

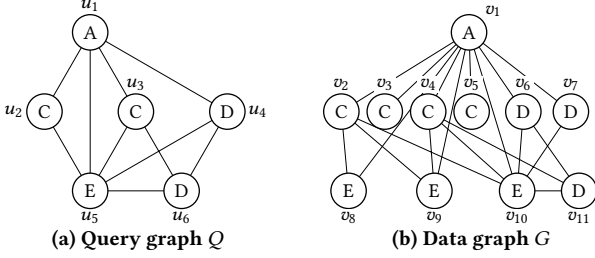


Figure 1: Running example.

Thus, Q_f contains all query vertices and exactly the query edges preserved under f . The number of missing query edges under f is defined as $\delta(f) = |E(Q) \setminus E(Q_f)| = |E(Q)| - |E(Q_f)|$. Then, we have the following definition.

Definition 2 (θ -Similarity Match). Given a query graph Q , a data graph G , and an integer threshold θ , an injective, label-preserving mapping $f : V(Q) \rightarrow V(G)$ is a θ -similarity match of Q in G if Q_f is connected and $\delta(f) \leq \theta$.

Consider Figure 1 with $\theta = 2$. For $f_1 = \{u_1 \mapsto v_1, u_2 \mapsto v_2, u_3 \mapsto v_4, u_4 \mapsto v_7, u_5 \mapsto v_{10}, u_6 \mapsto v_{11}\}$ and $f_2 = \{u_1 \mapsto v_1, u_2 \mapsto v_2, u_3 \mapsto v_3, u_4 \mapsto v_6, u_5 \mapsto v_{10}, u_6 \mapsto v_{11}\}$, the missing query edges are $\{(u_4, u_6)\}$ and $\{(u_3, u_5), (u_3, u_6)\}$, respectively. Both Q_{f_1} and Q_{f_2} are connected, with $\delta(f_1) = 1$ and $\delta(f_2) = 2$, and both mappings are 2-similarity matches.

Because Q is connected, a mapping is a 0-similarity match iff it is an exact match of Q in G . For any θ -similarity match f , Q_f is the unique maximum spanning subgraph of Q exactly matched by f . Following the maximal-output semantics of prior work [41], we therefore report only f , which uniquely determines Q_f . Thus, two reported matches are distinct if and only if their mappings differ.

Problem Statement. Given a query graph Q , a data graph G , and an integer threshold θ , the *subgraph similarity matching problem* is to enumerate $\mathcal{M}_\theta(Q, G) = \{f \mid f \text{ is a } \theta\text{-similarity match of } Q \text{ in } G\}$.

2.2 Existing Relaxation-Based Framework

Existing methods for subgraph similarity matching generally follow the relaxation-based framework [16, 36, 38, 41, 42], as summarized in Algorithm 1. The framework consists of three stages: query relaxation, exact matching, and verification and refinement.

In the *query relaxation stage*, the query graph Q is transformed into a collection $\mathcal{R}$ of relaxed structures that collectively cover the admissible missing-edge cases. Depending on the method, these structures may be edge-deleted query subgraphs, spanning trees, seed graphs, or local patterns obtained through query decomposition. A relaxed structure may encode multiple missing-edge cases rather than correspond to a single set of deleted edges.

In the *exact matching stage*, exact subgraph matching techniques [1, 3, 8] are applied to the relaxed structures, producing a collection $\mathcal{C}$ of candidate mappings or partial mappings that may be generated multiple times across different structures.

In the *verification-and-refinement stage*, the candidates are extended, combined, or refined as required by the method. The resulting mappings are checked against Q for injectivity, label preservation, connectivity of the preserved query graph, and satisfaction of the missing-edge threshold. Duplicate mappings are then removed.

Algorithm 1: Existing Relaxation-Based Framework

Input: a query graph Q , a data graph G , and a threshold θ

Output: all mappings in $\mathcal{M}_\theta(Q, G)$

- 1 $\mathcal{R} \leftarrow \text{GenerateRelaxedStructures}(Q, \theta)$
 - 2 $\mathcal{C} \leftarrow \text{ExactMatch}(\mathcal{R}, G)$
 - 3 $\mathcal{M} \leftarrow \text{VerifyAndRefine}(Q, G, \mathcal{C}, \theta)$
 - 4 **return** $\mathcal{M}$
-

Existing methods [16, 36, 38, 41, 42] differ primarily in (i) how they construct and represent relaxed structures, (ii) how they share computation across the associated exact-matching tasks, and (iii) how they assemble, verify, and deduplicate intermediate results. Despite these differences, relaxed query structures and their exact matches remain the basic units of computation. *The search is therefore organized around representations of missing-edge choices, whereas the outputs are vertex mappings.* Instead, our proposed method removes this intermediate representation and directly enumerates mappings, as described in Section 3.

3 ANCHOR-FRONTIER ENUMERATION

To eliminate the mismatch between relaxation-based search and vertex-mapping outputs, we propose *Anchor-Frontier Enumeration* (AFE), a mapping-centric framework that directly enumerates injective, label-preserving partial vertex mappings. Rather than constructing relaxed query structures, AFE determines preserved and missing query edges incrementally as a partial mapping grows and prunes a branch once its accumulated missing-edge cost exceeds θ . By aligning the search with the output representation, AFE eliminates repeated exploration through overlapping relaxations and avoids post-hoc cross-structure deduplication.

Direct mapping enumeration, however, raises a central challenge: maintaining the connectivity of the preserved query graph. Under the missing-edge model, a fixed connected query order does not guarantee that the next query vertex has a preserved edge to any mapped query vertex, since it may become connected only through other unmapped vertices. AFE addresses this challenge through an *anchor-frontier mechanism*, which repeatedly expands the current mapping through frontier vertices along selected anchor edges, thereby inducing an adaptive query-vertex expansion order during search. This design enables complete and duplicate-free enumeration while retaining neighborhood-based candidate generation and exact missing-edge accounting. We next present the resulting anchor-edge expansion.

3.1 Anchor-Edge Expansion

Many exploration-based subgraph matching algorithms use a connected matching order, so every query vertex except the root has a previously mapped neighbor that serves as a pivot for candidate generation [2, 14, 25]. The candidates of a new query vertex can then be restricted to the neighborhood of the mapped data vertex.

Under the missing-edge model, however, the edge to such a pivot may be missing in a feasible mapping. Restricting the new query vertex to the pivot neighborhood would miss valid mappings, whereas scanning its entire candidate set would lose the benefit of neighborhood-based expansion. AFE avoids both problems by

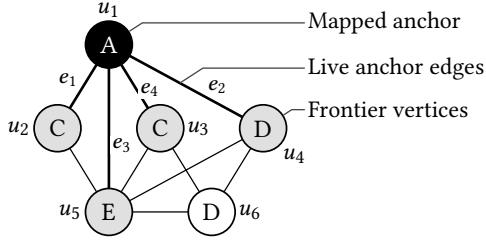


Figure 2: Anchor-frontier state S_0 .

selecting anchor edges according to the current search state and extending frontier vertices through those that are preserved.

Search state. A search state is represented by $S = (M, c, X)$, where M is the current partial mapping, c is the accumulated missing-edge cost, and X is the *excluded-edge set* containing query edges committed to be missing by previous exclusion decisions. Here, c is the cost already charged in the current state: each edge in X contributes one unit when excluded, while any other missing query edge is charged when its second endpoint is mapped. Let $V_M = \text{dom}(M)$ denote the mapped query vertices, and let $\text{im}(M) = \{M(u) \mid u \in V_M\}$ denote the data vertices already used by M .

Definition 3 (Frontier Vertex and Anchor Edge). Given a state $S = (M, c, X)$, its query frontier is $F(S) = \{u \in V(Q) \setminus V_M \mid N_Q(u) \cap V_M \neq \emptyset\}$. For a frontier vertex $u \in F(S)$, a vertex $a \in N_Q(u) \cap V_M$ is an *anchor* of u , and (u, a) is an *anchor edge*. An anchor edge is *live* in S if it does not belong to X .

Example 1. Consider the current search state $S_0 = (M_0, 0, \emptyset)$ shown in Figure 2, where $M_0 = \{u_1 \mapsto x_1\}$. Here and in the following examples, $x_i \in V(G)$ denotes a data vertex to which u_i is mapped. Since $V_{M_0} = \{u_1\}$, its unmapped neighbors u_2, u_3, u_4, u_5 form the query frontier $F(S_0) = \{u_2, u_3, u_4, u_5\}$. The vertex u_6 is not in the frontier because it has no neighbor in V_{M_0} . For every frontier vertex, u_1 is its anchor. Because $X = \emptyset$, all four anchor edges are live; Figure 2 labels them as $e_1 = (u_2, u_1)$, $e_2 = (u_4, u_1)$, $e_3 = (u_5, u_1)$, and $e_4 = (u_3, u_1)$.

Let H be a connected component of $Q[V(Q) \setminus V_M]$. Any feasible mapping extending M can connect H to the mapped query vertices only through an edge between $V(H)$ and V_M . Because the preserved query graph must be connected, it must preserve at least one live anchor edge between $V(H)$ and V_M .

AFE branches on live anchor edges. For a selected anchor edge $e = (u, a)$, where $u \notin V_M$ and $a \in V_M$, we have two branches:

- **Inclusion.** AFE preserves e and maps u to an unused data vertex v adjacent to $M(a)$. It then charges any newly missing query edges between u and its mapped neighbors.
- **Exclusion.** AFE marks e as missing, adds e to X , charges one unit to c , and leaves M unchanged.

The inclusion branch extends the mapping through a preserved anchor edge and therefore maintains connectivity of the preserved mapped query subgraph. The exclusion branch leaves M unchanged and continues with another live anchor edge. Repeated exclusions thus induce a sequential *exclusion chain*.

Exclusion chain. Consider a chain starting from $S = (M, c, X)$ and selecting live anchor edges $e_1, e_2, \dots$ in order. For each e_i , AFE

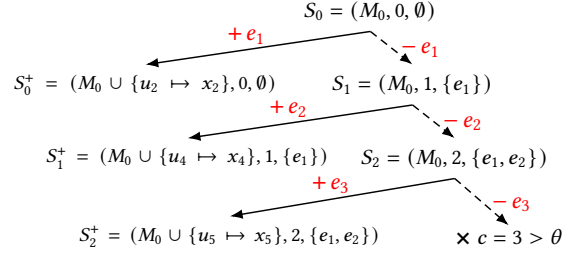


Figure 3: Exclusion-chain branching from S_0 .

first explores its inclusion branch. If exclusion remains feasible, AFE marks e_i as missing before proceeding to the next edge. Thus, when the inclusion branch of e_i is explored, $e_1, \dots, e_{i-1}$ have been excluded, while e_i is required to be preserved.

Example 2. Figure 3 shows the exclusion chain induced by the same state S_0 in Figure 2. In the figure, S_i denotes the chain state after excluding $e_1, \dots, e_i$, and S_i^+ denotes its inclusion successor that preserves e_{i+1} . With $\theta = 2$, AFE considers the anchor-edge prefix $E^* = \langle e_1, e_2, e_3 \rangle$. Its three inclusion branches preserve e_1, e_2, e_3 after 0, 1, 2 preceding exclusions, respectively, and map the corresponding frontier vertices as shown in the figure. A third exclusion would increase the cost to $3 > \theta$, so the chain stops before e_4 .

The length of an exclusion chain is bounded by both the remaining missing-edge budget and connectivity. Let $b = \theta - c$ be the remaining budget at the start of the chain. Since each exclusion consumes one unit of budget, at most b selected edges can be excluded. It therefore suffices to consider at most $b + 1$ anchor edges; for the $(b + 1)$ -st edge, only the inclusion branch can be feasible. Connectivity may terminate the chain earlier. If the selected edges exhaust all live anchor edges between an unmapped component of $Q[V(Q) \setminus V_M]$ and V_M , they cannot all be missing in any feasible mapping extending the current state. AFE therefore explores their inclusion branches but does not continue with the state in which all of them are excluded.

The anchor-edge expansion provides two main benefits. First, every mapping extension proceeds through a preserved anchor edge, so candidates can be obtained from the neighborhood of an already mapped data vertex rather than by scanning the entire candidate set. Second, the exclusion chain partitions feasible mappings by the first selected anchor edge they preserve, preventing the same θ -similarity match from being generated through multiple relaxation choices. The completeness and duplicate-free properties of this branching scheme are formalized by Theorem 1.

3.2 Overall Procedure

Algorithm 2 summarizes the AFE procedure. AFE performs a single depth-first search over partial vertex mappings, maintaining the missing-edge cost and exclusion decisions incrementally. Specifically, AFE iteratively conducts the following operations:

- Select a live anchor edge;
- Explore the branch that preserves the edge and maps its frontier vertex; and

- If the edge can be excluded, commit it to be missing, add it to X , charge one unit to c , and continue with the next live anchor edge.

An exclusion chain terminates before violating either the missing-edge budget or the connectivity condition.

Before enumeration, AFE invokes `DeletionAwareFilter`, which applies the neighborhood-aggregate filter of `TreeSpan` [41] to create a candidate set $C(u)$ for each query vertex u (Line 2). For each query-data vertex pair, the filter compares neighborhood label frequencies and removes the pair if the aggregate deficit exceeds θ . AFE selects the root $r = \arg \min_{u \in V(Q)} |C(u)|/d_Q(u)$, following the heuristic in [2] that favors fewer candidates and higher degree (Line 3). For each $v \in C(r)$, AFE initializes the search with $r \mapsto v$ (Line 5).

Consider a recursive call $\text{Search}(M, c, X)$. If all query vertices are mapped, AFE reports the current mapping M (Lines 8–10). Otherwise, `SelectAnchorEdges` constructs an ordered prefix E^* of live anchor edges for the current exclusion chain. The live anchor edges are ranked using the heuristic in Section 5.1, and the prefix is bounded by the remaining budget and the connectivity condition described above (Line 11). AFE processes the edges in E^* sequentially (Lines 12–21). When processing $e_i = (u, a)$, the preceding edges $e_1, \dots, e_{i-1}$ have already been excluded and incorporated into c and X , whereas e_i is required to be preserved in the current inclusion branch. Accordingly, AFE considers only unused candidates in $C(u)$ that are adjacent to $M(a)$ (Line 13). A candidate v is *feasible* only if it respects the exclusion decisions in X : for every mapped neighbor w of u with $(u, w) \in X$, we require $(v, M(w)) \notin E(G)$ (Line 14). For each feasible v , `IncrementalCost` counts newly missing query edges between u and its mapped neighbors, excluding edges already charged through X (Line 15). If the updated cost remains within θ , AFE recursively explores the extended mapping $M \cup \{u \mapsto v\}$ (Lines 16–17). After completing the inclusion branch of e_i , AFE considers its exclusion branch. If $c + 1 > \theta$, the exclusion would exceed the budget; because every subsequent branch on the same chain presupposes the exclusion of e_i , the procedure returns. Otherwise, AFE adds e_i to X , increments c , and continues with the next anchor edge (Lines 18–21).

Theorem 1 (Correctness of AFE). Algorithm 2 enumerates every mapping in $\mathcal{M}_\theta(Q, G)$ exactly once.

The proof of Theorem 1 is provided in Appendix A.1.

4 COST-SPLIT COMPRESSION

4.1 Motivation

AFE eliminates structure-level redundancy, but its inclusion branches still create one successor state for each feasible candidate of a frontier vertex. For a selected anchor edge (u, a) , AFE considers each feasible candidate v of u separately and extends the mapping with $u \mapsto v$. Before the choice of v affects later constraints, these branches differ only in the mapping of u and otherwise repeat the same subsequent search. This candidate-wise expansion therefore introduces *candidate-level redundancy*.

A natural way to avoid such premature branching is to defer selecting a concrete mapping for a query vertex. When several candidates can share the currently exposed search decisions, they can be represented collectively until a later dependency requires

Algorithm 2: AFE Enumeration

Input: a query graph Q , a data graph G , and a threshold θ
Output: all mappings in $\mathcal{M}_\theta(Q, G)$

```

1  $\mathcal{M} \leftarrow \emptyset$ 
2  $C \leftarrow \text{DeletionAwareFilter}(Q, G, \theta)$ 
3  $r \leftarrow \text{SelectRoot}(Q, C)$ 
4 foreach  $v \in C(r)$  do
5    $\text{Search}(\{r \mapsto v\}, 0, \emptyset)$ 
6 return  $\mathcal{M}$ 
7 Function  $\text{Search}(M, c, X)$ :
8   if  $|M| = |V(Q)|$  then
9      $\mathcal{M} \leftarrow \mathcal{M} \cup \{M\}$ 
10    return
11   $E^* \leftarrow \text{SelectAnchorEdges}(M, c, X, \theta)$ 
12  foreach  $e_i = (u, a) \in E^*$  do
13    foreach  $v \in C(u) \cap N_G(M(a)) \setminus \text{im}(M)$  do
14      if  $v$  is feasible with respect to  $(M, X)$  then
15         $\Delta \leftarrow \text{IncrementalCost}(u, v, M, X)$ 
16        if  $c + \Delta \leq \theta$  then
17           $\text{Search}(M \cup \{u \mapsto v\}, c + \Delta, X)$ 
18      if  $c + 1 > \theta$  then
19        return
20       $c \leftarrow c + 1$ 
21       $X \leftarrow X \cup \{e_i\}$ 

```

them to be distinguished. This deferred-mapping idea has been explored in exact subgraph matching [13, 17, 18, 21, 34]. For example, CEMR [34] introduces a black-white aggregated representation, where a black query vertex is mapped to a single data vertex, whereas a white query vertex retains a candidate set that can be processed collectively. However, under the missing-edge model, candidates may incur different missing-edge costs and therefore cannot always share one search state.

To resolve this tension, we develop *Cost-Split compression*, which couples deferred mapping with exact cost partitioning. Cost-Split aggregates only feasible candidates with the same incremental missing-edge cost and charges their common cost immediately, while leaving the concrete mapping unresolved. When later dependencies distinguish candidates by feasibility or incremental cost, Cost-Split filters or partitions only the affected candidate set. Cost-Split replaces AFE’s candidate-wise inclusion expansion with black or white expansion while leaving the anchor-edge exclusion chain unchanged. We next define the black-white compressed state (Section 4.2), present the two expansion modes and terminal completion (Section 4.3), and give the complete Cost-Split procedure (Section 4.4). Together, these components reduce candidate-level redundancy while preserving AFE’s exact missing-edge accounting and duplicate-free enumeration.

4.2 Black–White Cost-Split State

Following the black–white abstraction of CEMR [34], Cost-Split distinguishes concrete and unresolved query vertices while associating each search state with a single exact missing-edge cost. Every query vertex is in exactly one of the following three states:

- (1) A *black vertex* is mapped to a single data vertex.
- (2) A *white vertex* has been processed by an inclusion branch but is not yet mapped to a single data vertex. Instead, a nonempty candidate set is retained for it.
- (3) An *unprocessed vertex* has neither a concrete mapping nor a retained candidate set.

A Cost-Split search state is represented by $S = (M, W, c, X)$, where M is the current partial mapping of black vertices, W maps each white vertex to its nonempty candidate set, c is the accumulated missing-edge cost, and X is the excluded-edge set containing query edges committed to be missing by previous anchor-edge exclusion decisions. Let $V_B(S) = \text{dom}(M)$ and $V_W(S) = \text{dom}(W)$ denote the black and white query vertices, respectively. For each $u \in V_W(S)$, $W(u) \subseteq V(G)$ is the candidate set retained for u . The processed and unprocessed vertex sets are $V_P(S) = V_B(S) \cup V_W(S)$ and $V_U(S) = V(Q) \setminus V_P(S)$, respectively.

We extend the frontier and anchor terminology of Definition 3 to Cost-Split states by replacing the mapped vertex set V_M with the processed vertex set $V_P(S)$. Thus, a frontier vertex lies in $V_U(S)$ and has a neighbor in $V_P(S)$, which may serve as either a black or white anchor. The mapping of a white anchor is fixed before the anchor is used for candidate generation.

A concrete realization of S extends M by selecting one candidate from $W(u)$ for each white vertex u , subject to injectivity and the exclusion decisions in X whose endpoints have both been processed. A well-formed Cost-Split state satisfies the following properties:

- (1) **Exact state cost.** Every concrete realization of S has the same accumulated cost c : one unit for every edge in X , plus one unit for every missing nonexcluded query edge whose endpoints have both been processed.
- (2) **Uniform candidate cost.** For each white vertex u , all candidates in $W(u)$ incur the same missing-edge cost over the nonexcluded edges from u to its processed neighbors.
- (3) **No white–white edge.** No query edge has two white endpoints. Otherwise, whether the edge is missing would depend on the candidates selected for its two endpoints, and its cost could not yet be determined.

These properties allow one state to represent multiple concrete mappings with a single exact cost. Their preservation is formalized in Lemma 1.

4.3 Cost-Split Expansion and Completion

Cost-Split realizes each inclusion branch in two modes. *Black expansion* maps the frontier vertex to a single candidate and refines the existing white candidate sets affected by this mapping. *White expansion* keeps the frontier mapping unresolved and groups its candidates by exact incremental cost. Before either expansion, the selected anchor must be mapped to a single data vertex.

Fixing the Anchor Mapping. The preserved edge (u, a) can restrict the candidates of u only when a is mapped to a single data

vertex. If a is black, its mapping $M(a)$ is used directly. If a is white, Cost-Split branches over the unused candidates in $W(a)$, fixing a to one candidate and turning it black in each branch. No additional missing-edge cost is incurred because all query edges between a and previously processed vertices have already been accounted for.

Cost-Split then selects black or white expansion using the heuristic in Section 5.2 and applies the selected mode independently to each resulting state.

4.3.1 Black Expansion. Black expansion maps u to one feasible candidate at a time and refines the candidate sets of its white neighbors. For the preserved anchor edge (u, a) , the candidate set of u is $D_S(u, a) = (C(u) \cap N_G(M(a))) \setminus \text{im}(M)$, where $\text{im}(M)$ denotes the data vertices already used by M . The intersection with $N_G(M(a))$ preserves (u, a) , while removing $\text{im}(M)$ enforces injectivity. For each $v \in D_S(u, a)$, let $\Delta_S(u, v)$ denote the additional missing-edge cost incurred by mapping u to v with respect to the black neighbors of u . Edges in X must remain missing and are not charged again, while every other missing edge from u to a black neighbor contributes one unit of cost. A candidate is discarded if it violates X or if $c + \Delta_S(u, v) > \theta$.

Once u is mapped to v , the edges between u and its existing white neighbors become exposed. For a white neighbor $w \in N_Q(u) \cap V_W(S)$, the mapping $u \mapsto v$ splits $W(w)$ into the adjacent and nonadjacent groups $W_0(w) = \{x \in W(w) \mid (v, x) \in E(G)\}$ and $W_1(w) = \{x \in W(w) \mid (v, x) \notin E(G)\}$, respectively. The following formulas describe the first such refinement.

If $(u, w) \notin X$, each nonempty group produces a successor state. The adjacent group produces

$$(M \cup \{u \mapsto v\}, W[w \mapsto W_0(w)], c + \Delta_S(u, v), X),$$

where (u, w) is preserved without additional cost. The nonadjacent group produces

$$(M \cup \{u \mapsto v\}, W[w \mapsto W_1(w)], c + \Delta_S(u, v) + 1, X),$$

where (u, w) becomes missing. A state is created only when its candidate group is nonempty and its cost does not exceed θ .

If $(u, w) \in X$, only the nonadjacent group is feasible. Since (u, w) has already been charged as missing, the corresponding state retains $W_1(w)$ without adding another unit of cost.

The refinement is applied successively to all white neighbors of u . Each subsequent step starts from the current successor states, carrying forward the previous candidate-set restrictions and incurred costs; all other white candidate sets remain unchanged.

Example 3. Figure 4 shows black expansion. It starts from $S = (M_0, W, 0, \emptyset)$, where $M_0 = \{u_1 \mapsto x_1, u_4 \mapsto x_4\}$ and $W(u_2) = W_2$, $W(u_3) = W_3$. Black expansion selects a feasible candidate $x_5 \in C(u_5) \cap N_G(M_0(u_1))$ with $\Delta_S(u_5, x_5) = 0$, maps u_5 to x_5 , and sets $M' = M_0 \cup \{u_5 \mapsto x_5\}$. It then partitions W_2 and W_3 according to adjacency to x_5 . For $i \in \{2, 3\}$, candidates in $W_i \cap N_G(x_5)$ preserve (u_i, u_5) , whereas candidates in $W_i \setminus N_G(x_5)$ make it missing and add one unit of cost. Accordingly, the two signs in each successor-state label in Figure 4 indicate the retained groups for u_2 and u_3 , respectively. The four combinations yield successor states with costs 0, 1, 1, 2; X remains empty because these costs arise from exposed edge outcomes rather than anchor-edge exclusions.

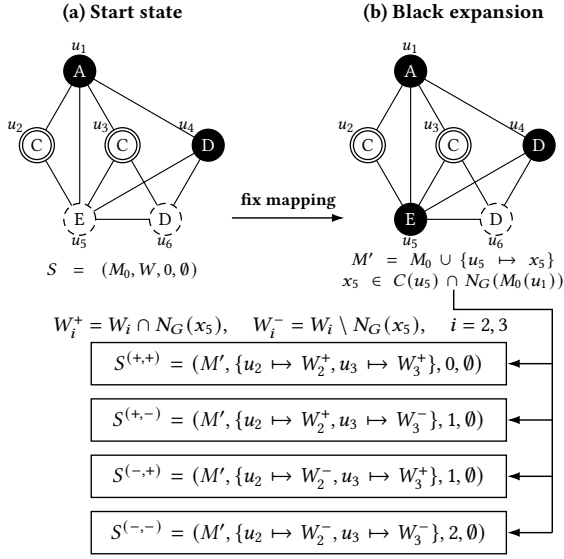


Figure 4: Black expansion at u_5 .

4.3.2 White Expansion and Initial Cost Partition. White expansion keeps the mapping of u unresolved. Before making u white, Cost-Split fixes the mappings of all vertices in $N_Q(u) \cap V_W(S)$ to mutually distinct unused candidates, so every processed neighbor of u is black. All other white candidate sets remain unchanged. Fixing these mappings does not change the accumulated cost because their edges to previously processed vertices have already been accounted for. Let S' denote any state obtained after these white-neighbor mappings have been fixed. Cost-Split forms the candidate set of u as $D_{S'}(u, a) = (C(u) \cap N_G(M_{S'}(a))) \setminus \text{im}(M_{S'})$, where $\text{im}(M_{S'})$ denotes the data vertices already used by $M_{S'}$. For each $v \in D_{S'}(u, a)$, $\Delta_{S'}(u, v)$ denotes the exact additional missing-edge cost incurred by mapping u to v in S' . An edge in $X_{S'}$ must remain missing and is not charged again, while every other missing edge from u to a processed neighbor contributes one unit of cost. Candidates that violate $X_{S'}$ or satisfy $c_{S'} + \Delta_{S'}(u, v) > \theta$ are discarded.

White expansion creates one successor state per exact incremental-cost group rather than one per candidate. For each feasible incremental cost d , Cost-Split forms $W_d(u) = \{v \in D_{S'}(u, a) \mid \Delta_{S'}(u, v) = d\}$. For each d with $W_d(u) \neq \emptyset$, it creates

$$S_d = (M_{S'}, W_{S'} \cup \{u \mapsto W_d(u)\}, c_{S'} + d, X_{S'}).$$

Hence, candidates with the same incremental cost share one successor state, while the concrete mapping of u remains unresolved.

Example 4. Figure 5 shows white expansion. It starts from the same state $S = (M_0, W, 0, 0)$ with $W(u_2) = W_2$ and $W(u_3) = W_3$. White expansion first fixes the mappings of u_2 and u_3 . For one feasible branch, choose mutually distinct, unused vertices $x_i \in W_i$ for $i \in \{2, 3\}$ and let $M' = M_0 \cup \{u_i \mapsto x_i : i = 2, 3\}$. The resulting state is $S' = (M', \emptyset, 0, 0)$. It then partitions the candidates of u_5 into $W_d(u_5)$, where $x_5 \in W_d(u_5)$ exactly when $x_5 \in D_{S'}(u_5, u_1)$ and $\Delta_{S'}(u_5, x_5) = d$. Each nonempty group yields $S_d = (M', \{u_5 \mapsto W_d(u_5)\}, d, 0)$. Figure 5 shows the groups for $d = 0, 1, 2$.

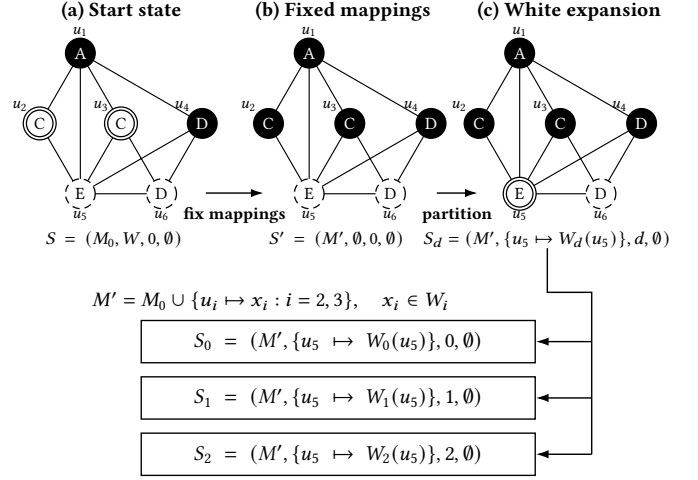


Figure 5: White expansion at u_5 .

4.3.3 Terminal White Enumeration. The anchor-frontier search terminates when every query vertex has been processed. If $V_W(S) = \emptyset$, M is a feasible mapping and can be reported directly. Otherwise, no query edge joins two white vertices, and all white-black edges have already been evaluated. Resolving the remaining white mappings therefore changes neither the accumulated cost nor the edge decisions. Cost-Split enumerates tuples $(v_u)_{u \in V_W(S)} \in \prod_{u \in V_W(S)} W(u)$ such that $M \cup \{u \mapsto v_u \mid u \in V_W(S)\}$ is injective, and reports each resulting complete mapping.

4.4 Cost-Split Expansion Procedure

Algorithm 3 summarizes the preserved-edge expansion used by AFEC. Given a live anchor edge (u, a) and a Cost-Split state $S = (M, W, c, X)$, it fixes the mapping of a white anchor when necessary, selects the execution mode, and generates the successor states in which (u, a) is preserved (Lines 1–6). The exclusion branch and exclusion-chain construction remain unchanged.

In black mode, the algorithm enumerates each feasible candidate v and invokes `RefineWhiteNeighbors` to propagate the newly exposed edge outcomes to adjacent white candidate sets (Lines 7–13). In white mode, it first fixes the mappings of the existing white neighbors of u , partitions the feasible candidates by their exact incremental costs, and creates one successor state for each nonempty bucket $W_d(u)$ (Lines 14–24). The auxiliary procedures implement the mapping choices and candidate-set refinements defined above. Both modes generate well-formed successor states with a single exact accumulated cost. Black expansion incurs candidate-wise branching immediately, whereas white expansion compresses candidates in each cost group into one successor state and leaves the mapping of u unresolved. The preferable mode therefore depends on the current state, motivating the dynamic black-white selection strategy in Section 5.2. When all query vertices have been processed, the remaining white candidate sets are resolved by Terminal White Enumeration as described in Section 4.3.3.

Lemma 1 (Lossless Cost-Split Expansion). Every Cost-Split state generated by AFEC is well formed. For any selected anchor edge

Algorithm 3: Cost-Split Preserved-Edge Expansion

```

1 Function ExpandPreservedEdge( $u, a, S$ ):
2   foreach  $S' \in \text{FixAnchorMapping}(a, S)$  do
3     if  $\text{SelectBWMode}(u, a, S') = \text{black}$  then
4        $\text{BlackExpand}(u, a, S')$ 
5     else
6        $\text{WhiteExpand}(u, a, S')$ 
7 Function BlackExpand( $u, a, S$ ):
8    $D \leftarrow (C(u) \cap N_G(M(a))) \setminus \text{im}(M)$ 
9   foreach  $v \in D$  do
10     $d \leftarrow \Delta_S(u, v)$ 
11    if  $v$  is feasible in  $S$  and  $c + d \leq \theta$  then
12      foreach  $S' \in \text{RefineWhiteNeighbors}(u, v, d, S)$ 
13        do
14           $\text{Search}(S')$ 
14 Function WhiteExpand( $u, a, S$ ):
15   foreach  $S' \in \text{FixWhiteNeighborMappings}(u, S)$  do
16      $D \leftarrow (C(u) \cap N_G(M_{S'}(a))) \setminus \text{im}(M_{S'})$ 
17     initialize an empty bucket  $W_d(u)$  for each
18       incremental cost  $d$ 
19     foreach  $v \in D$  do
20        $d \leftarrow \Delta_{S'}(u, v)$ 
21       if  $v$  is feasible in  $S'$  and  $c_{S'} + d \leq \theta$  then
22          $W_d(u) \leftarrow W_d(u) \cup \{v\}$ 
23       foreach  $d$  such that  $W_d(u) \neq \emptyset$  do
24          $S_d \leftarrow (M_{S'}, W_{S'} \cup \{u \mapsto W_d(u)\}, c_{S'} + d, X_{S'})$ 
25          $\text{Search}(S_d)$ 

```

required to be preserved, either expansion mode produces pairwise-disjoint successor states whose concrete realizations exhaust exactly the feasible concrete extensions of the current state that preserve this edge. Each successor state maintains the exact accumulated missing-edge cost. Terminal White Enumeration reports every concrete realization of a terminal state exactly once.

The proof of Lemma 1 is provided in Appendix A.2.

Corollary 1 (Correctness of AFEC). By Theorem 1 and Lemma 1, AFEC enumerates every mapping in $\mathcal{M}_\theta(Q, G)$ exactly once.

5 AUXILIARY TECHNIQUES

AFEC uses two state-adaptive heuristics to control local branching during enumeration. *Anchor-edge ordering* ranks the live anchor edges considered along each exclusion chain (Section 5.1), while *dynamic black–white selection* chooses between black and white expansion for the current inclusion branch (Section 5.2).

5.1 Dynamic Anchor-Edge Ordering

Along an exclusion chain, the ordering of live anchor edges determines which inclusion branches are explored within the prefix

Table 1: Statistics of the 10 real-world data graphs.

Dataset	Abbr.	$ V(G) $	$ E(G) $	d_G	$ \Sigma $	Type
Yeast	ys	3,112	12,519	8.0	71	Protein
Citeseer	cs	3,264	4,536	2.8	6	Citation
Human	hu	4,674	86,282	36.9	44	Protein
HPRD	hp	9,460	34,998	7.4	307	Protein
WordNet	wn	76,853	120,399	3.1	5	Lexical
Twitch	tw	168,144	6,797,557	80.9	21	Social
DBLP	db	317,080	1,049,866	6.6	15	Social
Eu2005	eu	862,664	16,138,468	37.4	40	Web
Youtube	yt	1,134,890	2,987,624	5.3	25	Social
Patent	up	3,774,768	16,518,947	8.8	20	Citation

allowed by the remaining missing-edge budget and the connectivity bound. AFEC therefore prioritizes anchor edges with smaller estimated candidate support, while favoring frontier vertices with more live anchors.

For a live anchor edge (u, a) at state S , let $\lambda_S(u)$ denote the number of live anchors of u . We assign (u, a) the ordering score:

$$w_S(u, a) = \frac{1}{\lambda_S(u)} \begin{cases} |C(u) \cap N_G(M(a))|, & a \in V_B(S), \\ \frac{|W(a)|}{|C(a)|} \sum_{x \in C(a)} |C(u) \cap N_G(x)|, & a \in V_W(S). \end{cases}$$

When a is black, the score uses the number of candidates of u supported by the concrete mapping of a . When a is white, its concrete mapping is unresolved, so the second case estimates this support by scaling the average support over $C(a)$ by the current white candidate-set size $|W(a)|$. The average-support term depends only on the filtered candidate sets and can be precomputed. The factor $1/\lambda_S(u)$ favors frontier vertices with more live anchors. For AFE, all anchors are mapped and only the first case applies. AFEC orders the live anchor edges by increasing $w_S(u, a)$. The procedure `SelectAnchorEdges` (Line 11 of Algorithm 2) then retains the prefix permitted by the remaining missing-edge budget and the connectivity bound described in Section 3.1.

5.2 Dynamic Black–White Selection

Black and white expansion incur different sources of branching. Black expansion maps u to a concrete candidate immediately and may split adjacent white candidate sets, whereas white expansion keeps the mapping of u unresolved but first fixes the mappings of its existing white neighbors. Cost-Split therefore selects between the two modes according to their estimated branching scales. The procedure `SelectBWMode` (Line 3 of Algorithm 3) makes this decision after the mapping of the selected anchor a has been fixed.

Let $n_S(u, a)$ denote the number of candidates $v \in D_S(u, a)$ that are feasible in S and satisfy $c + \Delta_S(u, v) \leq \theta$. If $n_S(u, a) = 0$, the inclusion branch is pruned. Let $s_S(u) = |\{w \in N_Q(u) \cap V_W(S) \mid (u, w) \notin X\}|$ be the number of white neighbors whose edge to u has not been excluded. For each feasible candidate of u , black expansion may split each such white candidate set into a preserved group and a missing group. We estimate its branching scale as $N_b(u, a) = n_S(u, a) 2^{s_S(u)}$. For white expansion, let $k_S(u, a)$ denote the number of distinct currently exposed incremental costs $\Delta_S(u, v)$ among these feasible candidates. White expansion first fixes the

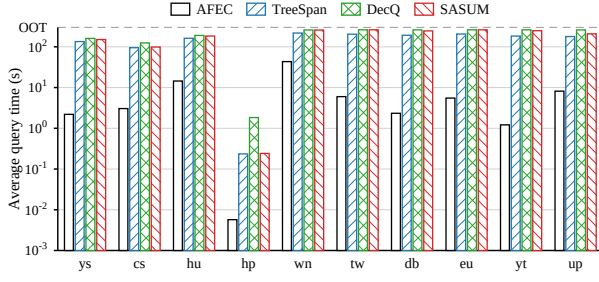


Figure 6: Average query time in the main comparison.

mappings of the existing white neighbors and then retains the candidates of u by cost group. We estimate its branching scale as $N_w(u, a) = k_S(u, a) \prod_{w \in N_Q(u) \cap V_W(S)} |W(w)|$.

Cost-Split selects white expansion when $N_w(u, a) \leq N_b(u, a)$, and selects black expansion otherwise. Thus, black expansion is favored when immediate candidate-wise branching is cheaper, whereas white expansion is favored when cost-wise aggregation offsets the overhead of fixing the white-neighbor mappings.

6 EXPERIMENTS

In this section, we conduct extensive experiments to evaluate the practical performance of AFEC. We compare AFEC with three state-of-the-art subgraph similarity matching algorithms¹: TreeSpan [41], DecQ [42], and SASUM [16], on 10 real-world graphs.

Setup. All algorithms are implemented in C++ and compiled with g++ 11.4.0 using -O3 and the C++17 standard. The experiments are conducted on a machine running Ubuntu 22.04.5 Linux, with an Intel 2.60 GHz CPU and 252 GB of main memory. We limit each query to 300 seconds and 64 GB. The source code of AFEC and our baseline implementations is available at <https://github.com/asdf1229/AFEC-SSM>.

Data graphs. Following existing studies [4, 34, 35] and surveys [12, 24, 39], We evaluate all algorithms on 10 real-world data graphs, which cover protein, citation, social, lexical, and Web networks with thousands to millions of vertices. Table 1 shows the statistics, where $d_G = 2|E(G)|/|V(G)|$ denotes the average degree and $|\Sigma|$ denotes the number of vertex labels.

Query graphs. Following prior subgraph matching studies [1, 24, 34], we generate query graphs by extracting connected induced subgraphs from the data graphs using a random-walk method. We consider four query sizes, $|V(Q)| \in \{8, 16, 24, 32\}$, and classify queries as sparse if $d_Q = 2|E(Q)|/|V(Q)| < 3$ and dense otherwise. For each combination of query size and density, we generate 100 queries. Thus, each data graph is evaluated with $4 \times 2 \times 100 = 800$ queries, resulting in 8,000 queries across the 10 datasets.

Evaluation metrics. By default, the missing-edge threshold is $\theta = 2$, and enumeration terminates after finding 10^8 matches. A query is regarded as successful if it completes enumeration or reaches the prescribed output limit, and as failed if it exceeds either the time or memory limit. The running time is measured separately for

¹We implement AFEC and independently re-implement the three baselines: TreeSpan, DecQ, and SASUM. Because no public baseline implementations are available, our implementations follow the original papers. All implementations use the same input format, resource limits, and output-limit semantics.

Table 2: Successful-query rates (%) in the main comparison.

Dataset	AFEC	TreeSpan	DecQ	SASUM
Yeast	100.00	75.13	45.00	47.13
Citeseer	99.63	80.63	58.00	66.88
Human	95.75	59.88	29.88	31.88
HPRD	100.00	100.00	99.75	100.00
WordNet	87.25	44.38	1.38	1.50
Twitch	99.75	72.38	0.00	0.00
DBLP	99.75	61.63	1.00	11.25
Eu2005	99.63	58.13	0.00	0.00
Youtube	100.00	78.50	0.00	6.88
Patent	98.88	64.00	2.00	26.38
Overall	98.06	69.46	23.70	29.19

Table 3: Average peak memory usage (GB).

Dataset	AFEC	TreeSpan	DecQ	SASUM
Yeast	0.01	8.47	34.95	34.68
Citeseer	0.01	5.12	26.81	24.72
Human	0.01	7.53	42.12	38.43
HPRD	0.01	0.02	0.45	0.09
WordNet	0.11	7.92	49.08	58.84
Twitch	1.74	16.33	53.84	58.32
DBLP	0.06	12.68	46.51	50.25
Eu2005	0.68	13.22	42.81	36.58
Youtube	0.24	13.30	48.11	47.04
Patent	0.91	15.75	37.36	31.30
Overall	0.38	10.04	38.21	38.03

each query and excludes graph loading time. We report the average query time, where a successful query contributes its actual running time and a failed query is assigned 300 seconds. In the figures, *OOT* denotes the query exceeds the 300-second limit.

6.1 Comparison with Baselines

We first evaluate the efficiency and robustness of AFEC with TreeSpan, DecQ, and SASUM. Figure 6 reports the average query time, and Table 2 reports the successful-query rate. We consider both metrics because many baseline queries neither complete enumeration nor reach the prescribed output limit within the resource limits. We have the following three observations. First, AFEC is consistently faster than all baselines, averaging 9.61 seconds over the full workload, compared with 179.95, 233.89, and 219.63 seconds for TreeSpan, DecQ, and SASUM, respectively. Second, relative to the fastest baseline on each dataset, AFEC achieves speedups ranging from 5.1× on WordNet to 159.9× on Youtube. Even on HPRD, where nearly all algorithms successfully process all queries, AFEC remains 42.8× faster than the fastest baseline. Third, AFEC is substantially more robust: it successfully processes 7,845 queries, achieving an overall success rate of 98.06%, compared with 69.46%, 23.70%, and 29.19% for TreeSpan, DecQ, and SASUM, respectively. AFEC maintains a success rate above 95% on nine datasets and achieves 87.25% on WordNet, where TreeSpan, DecQ, and SASUM achieve 44.38%, 1.38%, and 1.50%, respectively. Because failed queries contribute 300 seconds to the average, successful-query rates provide a complementary measure of robustness.

These results are consistent with three aspects of AFEC’s design. First, its unified enumeration framework searches directly over

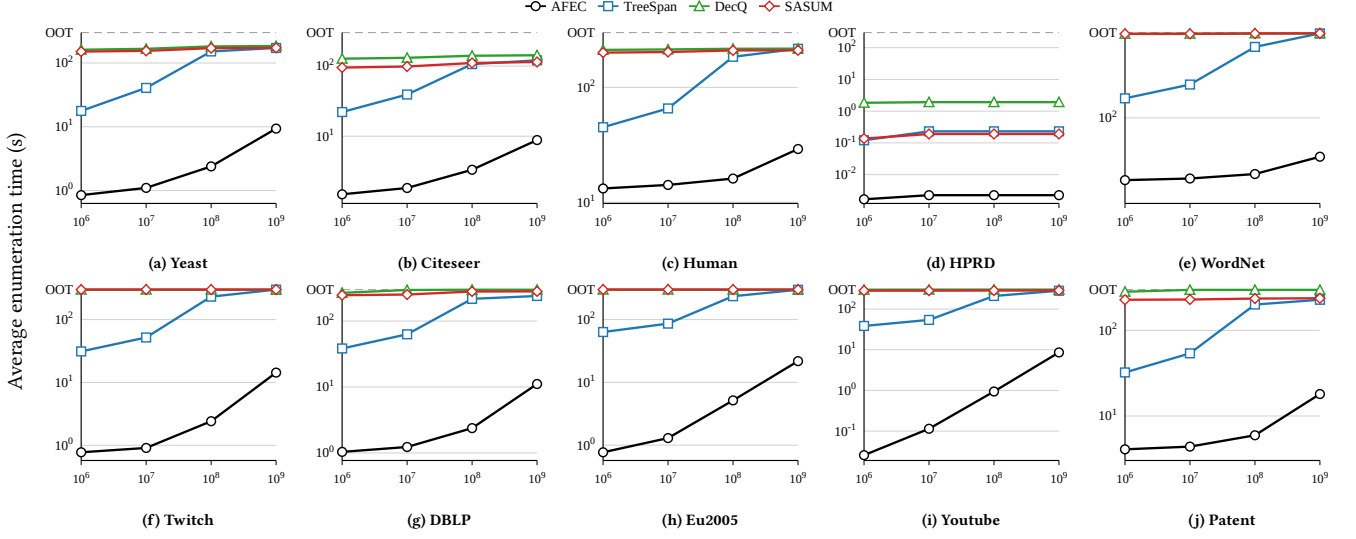


Figure 7: Effect of the output limit on average enumeration time after preprocessing.

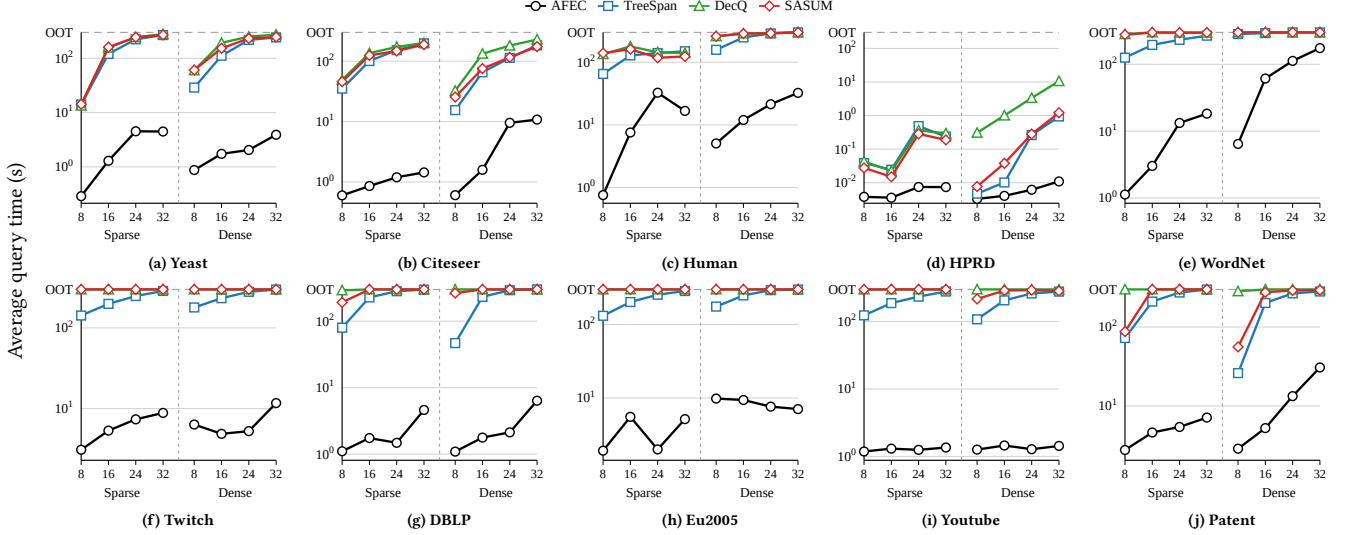


Figure 8: Effect of query size and density on average query time.

vertex mappings, avoiding repeated exploration through relaxed query structures and subsequent deduplication. Second, Cost-Split retains candidates with the same incremental missing-edge cost in a shared white state, allowing them to share subsequent exploration. Third, dynamic black-white selection adapts the expansion mode to the current state, while dynamic anchor-edge ordering prioritizes edges expected to produce smaller inclusion branches. Together, these techniques reduce both structure-level and candidate-level redundancy while avoiding ineffective branching.

Peak Memory Consumption. Table 3 shows that AFEC uses the least memory on all 10 datasets. Each per-query peak resident-set size is capped at 64 GB before averaging. Over the full workload, AFEC uses 0.38 GB on average, compared with 10.04 GB for

TreeSpan, 38.21 GB for DecQ, and 38.03 GB for SASUM. Its average peak memory remains below 2 GB on every dataset, whereas 2,367 DecQ queries and 2,432 SASUM queries reach the 64-GB reporting ceiling. These results are consistent with AFEC’s design: its unified search avoids materializing multiple relaxed query structures and large collections of overlapping intermediate matches, while Cost-Split retains candidates with the same exposed cost in a shared state until their concrete mappings are required.

6.2 Varying the Settings

Effect of Output Limit. We vary the output limit over 10^6 , 10^7 , 10^8 , and 10^9 matches on the full 8,000-query workload while fixing $\theta = 2$. Figure 7 reports the average enumeration time after

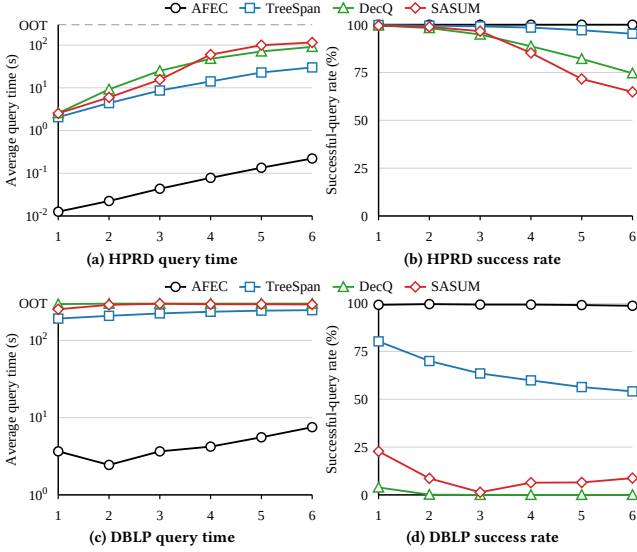


Figure 9: Effect of the missing-edge threshold on HPRD and DBLP.

preprocessing. Specifically, enumeration time increases with the output limit because each algorithm must produce more matches before reaching a checkpoint. AFEC remains faster than all baselines across the tested limits, whereas the baseline curves frequently approach OOT as the limit increases. Two dataset-specific patterns are noteworthy. On HPRD, the curves stabilize beyond an output limit of 10^7 , indicating that most queries complete full enumeration before the higher limits become active. On WordNet, by contrast, AFEC already incurs a relatively high cost at the smallest limit, while the baselines remain close to OOT. This pattern suggests that structural search difficulty, rather than output volume, dominates the WordNet workload. The upward curvature of some AFEC lines reflects the $10\times$ increase in requested output at each step and does not imply an increasing cost per result. Likewise, a baseline curve that remains nearly flat around OOT reflects the 300-second penalty for unsuccessful queries rather than insensitivity to the output limit.

Effect of Query Size and Density. For each dataset, we group queries by size $|V(Q)| \in \{8, 16, 24, 32\}$ and density, with 100 queries in each group. We fix $\theta = 2$ and the output limit at 10^8 matches. Figure 8 reports the average query times. AFEC is the fastest algorithm for every combination of dataset, query size, and query density. For sparse queries, its running time grows moderately with query size, while its successful-query rate remains close to 100%. The baselines, in contrast, become slower and less successful as query size increases. The size effect is stronger for dense queries, which contain more edge constraints and missing-edge decisions. AFEC remains at the millisecond level on HPRD and is nearly insensitive to query size and density on Youtube, whereas dense WordNet queries become sharply more difficult as their size grows.

Effect of the Missing-Edge Threshold. We vary the missing-edge threshold from 1 to 6 on HPRD and DBLP, using the same 800 queries at each threshold. The output limit is fixed at 10^8 matches.

Figure 9 reports the average query time and successful-query rate. We can that: 1) The performance gap generally widens as θ increases; 2) A larger threshold admits more missing-edge decisions and consequently expands the search space. On HPRD, all four algorithms perform well at small thresholds, but their performance diverges as the threshold grows. AFEC remains below one second and successfully processes every query throughout the tested range, whereas the baselines become slower and less successful. DBLP is more challenging: AFEC remains below 8 seconds and processes nearly all queries successfully at every threshold, while TreeSpan deteriorates steadily and DecQ and SASUM remain close to OOT with low successful-query rates. The DBLP curves are not strictly monotonic because a larger threshold can produce enough matches to reach the output limit earlier, locally reducing the measured time or increasing the successful-query rate.

6.3 Ablation Studies

We report one framework-level comparison and three component-level ablations. The framework-level comparison contrasts AFE with AFE-NoAF; the component-level ablations assess Cost-Split compression, dynamic black-white selection, and dynamic anchor-edge ordering. Figure 10 reports these four comparisons. Because AFE-NoAF has a high failure rate, Figure 10(a) reports both average time and successful-query rate, whereas the other panels report average time.

Effect of Anchor-Frontier Expansion. Figure 10(a) compares AFE with AFE-NoAF, which disables anchor-frontier expansion.² Without anchor-frontier expansion, AFE-NoAF branches over unused candidates without restricting extensions to mapped anchor neighborhoods and checks preserved-query connectivity only after a complete mapping is formed. Overall, AFE improves the successful-query rate from 21.90% to 94.26% and achieves a lower average time on every dataset. Even on HPRD, where AFE-NoAF still succeeds on 93.88% of the queries, its average time is 24.73 seconds, compared with 0.023 seconds for AFE. These results show that anchor-frontier expansion controls combinatorial growth by extending mappings through preserved anchor edges, restricting candidates to anchor neighborhoods, and pruning states that cannot maintain preserved-query connectivity.

Effect of Cost-Split Compression. Figure 10(b) compares AFEC with AFE, which disables Cost-Split but retains all other shared settings. AFE has $3.8\times$ AFEC’s overall average time, with the largest penalties on demanding workloads and little difference on HPRD. This result confirms that Cost-Split reduces candidate-level redundancy by retaining candidates with the same incremental missing-edge cost in a shared white state.

Effect of Dynamic Black-White Selection. Figure 10(c) compares AFEC with AFEC-White and AFEC-Black, which always use white and black expansion, respectively. These fixed policies are $1.3\times$ and $4.1\times$ slower overall. Although AFEC-White is slightly faster on Human and Twitch, no fixed mode performs best across all datasets. Dynamic selection achieves the best overall performance by adapting the expansion mode to each search state.

²Both variants disable Cost-Split and use the same candidate filtering and root selection; AFE-NoAF follows a fixed query-vertex order.

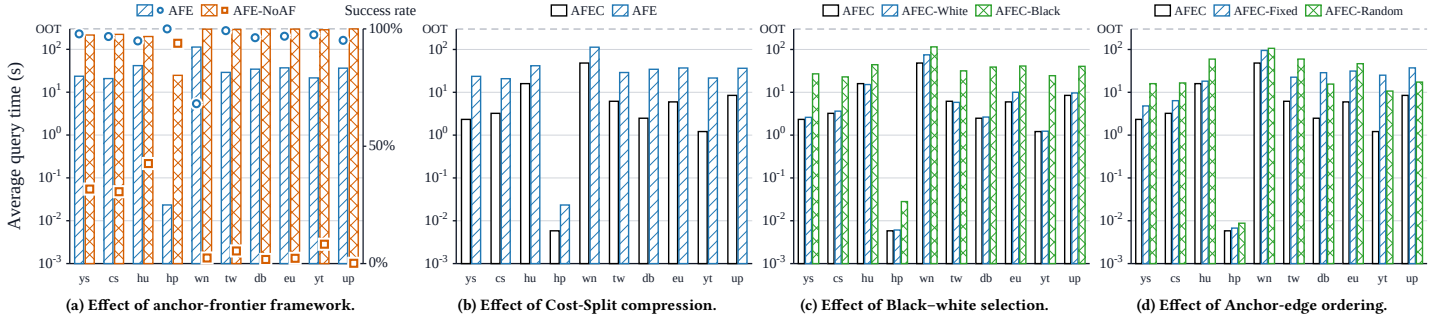


Figure 10: Ablation studies.

Effect of Dynamic Anchor-Edge Ordering. Figure 10(d) compares AFEC with AFEC-Fixed and AFEC-Random. AFEC-Fixed uses the priorities adopted by TreeSpan [41], whereas AFEC-Random uses a random order. They are 2.9 $\times$ and 3.7 $\times$ slower overall, respectively, and AFEC is faster on every dataset. These results show that dynamic anchor-edge ordering improves the search by prioritizing anchor edges expected to produce smaller inclusion branches.

Overall, the ablation results show that the four components address complementary sources of search cost: anchor-frontier expansion controls the mapping search space, Cost-Split compresses candidate-wise branches, dynamic black-white selection chooses the expansion mode, and dynamic anchor-edge ordering improves the exclusion-chain partition.

7 RELATED WORK

Subgraph similarity matching. Under the missing-edge model, subgraph similarity matching maps all query vertices but allows a bounded number of query edges to be missing, provided that the preserved query graph remains connected. Early approaches considered graph indexing and graph-edit-distance formulations [27, 33, 37]. SAPPER [38] enumerates query subgraphs obtained by deleting edges, whereas TreeSpan [41] represents feasible relaxations using spanning trees. DecQ [42] and SASUM [16] reuse intermediate matching results through query decomposition and a seed-graph lattice, respectively. Distributed methods organize relaxed queries using either h -tree decompositions [36] or prototypes associated with edit bounds [22]. Although these methods differ in how they represent relaxations and reuse exact matches, they all use relaxed query structures as the unit of search or composition. In contrast, AFE directly enumerates vertex mappings and accounts for missing query edges incrementally within a single search.

More recently, S^3 AND [30] searches for connected data subgraphs subject to aggregate neighbor-difference and keyword constraints. Under exact label matching and an appropriate structural-score bound, our missing-edge model can be formulated as a restricted case of S^3 AND. However, S^3 AND returns an induced data subgraph, which may contain edges absent from the query. Our model instead requires the preserved query graph to remain connected and reports the unique maximal match determined by each feasible mapping.

Exact subgraph matching. Exact subgraph matching enumerates all embeddings that preserve every query edge. Representative

backtracking-based algorithms include Ullmann’s algorithm [28], VF2 [5], and TurboISO [9]. Recent methods focus on reducing redundant computation during enumeration. CFL-Match [3] postpones Cartesian products across loosely coupled query parts. DAF [8] and GuP [1] reuse failure information to avoid revisiting unpromising search states. VEQ [15] and FiPE [19] exploit equivalence among query vertices or data candidates. Circinus [13] compresses candidate sets for query vertices outside a vertex cover and delays decomposition until dependencies require concrete mappings. CBF [21] uses vertex-cover-based compression to reduce the storage footprint of the instance set. BEE [40] uses block-separator decomposition to detect redundant searches at both global and partial-search levels. Recent surveys provide comprehensive overviews of exact subgraph matching techniques [12, 24, 39].

Our Cost-Split technique (Section 4) draws on two ideas developed for exact matching. First, *branch merging* postpones assignments so that multiple candidates can share subsequent extensions. CEMR [34] uses black-white aggregated embeddings and a cost model to select beneficial encodings. Second, *candidate-domain compression* keeps candidate sets unresolved and refines them during search. BSX [18] represents similar candidates along one dimension of a multidimensional search box while progressively refining the remaining dimensions. Unlike these exact-matching techniques, edit-aware enumeration must maintain a single exact missing-edge cost for every compressed state. Cost-Split therefore aggregates only candidates with the same incremental cost and refines a white candidate set when later dependencies distinguish its members by feasibility or incremental cost.

8 CONCLUSION

In this paper, we proposed Anchor-Frontier Enumeration (AFE), a mapping-centric framework that directly enumerates vertex mappings with incremental missing-edge accounting while ensuring connectivity and complete, duplicate-free enumeration. We further developed Cost-Split compression, dynamic anchor-edge ordering, and dynamic black-white selection to reduce candidate-level redundancy and ineffective branching. Together, these techniques form our AFEC algorithm. Experiments on 10 real-world graphs demonstrated substantial improvements in efficiency and memory usage over existing methods. Future work will extend the framework to richer edit models involving vertex and label differences.

REFERENCES

- [1] Junya Arai, Yasuhiro Fujiwara, and Makoto Onizuka. 2023. GuP: Fast subgraph matching by guard-based pruning. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.
- [2] Bibek Bhattarai, Hang Liu, and H Howie Huang. 2019. CECI: Compact embedding cluster index for scalable subgraph matching. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1447–1462.
- [3] Fei Bi, Lijun Chang, Xuemin Lin, Lu Qin, and Wenjie Zhang. 2016. Efficient subgraph matching by postponing cartesian products. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1199–1214.
- [4] Yunyoung Choi, Kunsoo Park, and Hyunjoon Kim. 2023. BICE: Exploring compact search space by using bipartite matching and cell-wide verification. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2186–2198.
- [5] Luigi P Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. A (sub) graph isomorphism algorithm for matching large graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26, 10 (2004), 1367–1372.
- [6] Michael R Garey and David S Johnson. 2002. *Computers and Intractability*. Vol. 29. wh freeman New York.
- [7] Michelle Girvan and Mark EJ Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826.
- [8] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1429–1446.
- [9] Wook-Shin Han, Jinsoo Lee, and Jeong-Hoon Lee. 2013. Turboiso: Towards ultrafast and robust subgraph isomorphism search in large graph databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 337–348.
- [10] G Traver Hart, Arun K Ramani, and Edward M Marcotte. 2006. How complete are current yeast and human protein-interaction networks? *Genome Biology* 7, 11 (2006), 120.
- [11] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. 2021. Knowledge graphs. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–37.
- [12] Haolin Jiang, Santosh Pandey, and Hang Liu. 2025. A comprehensive survey of subgraph matching: [Experiments & Analysis]. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–30.
- [13] Tatiana Jin, Boyang Li, Yichao Li, Qihui Zhou, Qianli Ma, Yunjian Zhao, Hongzhi Chen, and James Cheng. 2023. Circinus: Fast redundancy-reduced subgraph matching. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
- [14] Alpár Jüttner and Péter Madarasi. 2018. VF2++—An improved subgraph isomorphism algorithm. *Discrete Applied Mathematics* 242 (2018), 69–81.
- [15] Hyunjoon Kim, Yunyoung Choi, Kunsoo Park, Xuemin Lin, Seok-Hee Hong, and Wook-Shin Han. 2021. Versatile equivalences: Speeding up subgraph query processing and subgraph matching. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 925–937.
- [16] Song-Hyon Kim, Inchul Song, Kyong-Ha Lee, and Yoon-Joon Lee. 2013. SASUM: A sharing-based approach to fast approximate subgraph matching for large graphs. *IEEE Transactions on Information and Systems* 96, 3 (2013), 624–633.
- [17] Ziming Li, Youhuan Li, Xinhuan Chen, Lei Zou, Yang Li, Xiaofeng Yang, and Hongbo Jiang. 2024. NewSP: A new search process for continuous subgraph matching over dynamic graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 3324–3337.
- [18] Yujie Lu, Zhijie Zhang, and Weiguo Zheng. 2025. BSX: Subgraph matching with batch backtracking search. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.
- [19] Yujie Lu, Zhijie Zhang, Weiguo Zheng, and Lei Zou. 2025. Accelerating subgraph matching through fine-grained and powerful equivalences. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3896–3909.
- [20] Sadeh M. Milajerdi, Birhanu Eshete, Rigel Gjomemo, and V. N. Venkatakrishnan. 2019. POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1813–1830.
- [21] Miao Qiao, Hao Zhang, and Hong Cheng. 2017. Subgraph matching: on compression and computation. *Proceedings of the VLDB Endowment* 11, 2 (2017), 176–188.
- [22] Tashin Reza, Matei Ripeanu, Geoffrey Sanders, and Roger Pearce. 2020. Approximate pattern matching in massive graphs with precision and recall guarantees. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*. 1115–1131.
- [23] Ulrich Stelzl, Uwe Worm, Maciej Lalowski, Christian Haenig, Felix H Brembeck, Heike Goehler, Martin Stroedicke, Martina Zenkner, Anke Schoenherr, Susanne Koepfen, et al. 2005. A human protein-protein interaction network: A resource for annotating the proteome. *Cell* 122, 6 (2005), 957–968.
- [24] Shixuan Sun and Qiong Luo. 2020. In-memory subgraph matching: An in-depth study. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1083–1098.
- [25] Shixuan Sun, Xibo Sun, Yulin Che, Qiong Luo, and Bingsheng He. 2020. Rapid-match: A holistic approach to subgraph query processing. *Proceedings of the VLDB Endowment* 14, 2 (2020), 176–188.
- [26] Yuanyuan Tian, Richard C Mceachin, Carlos Santos, David J States, and Jignesh M Patel. 2007. SAGA: a subgraph matching tool for biological graphs. *Bioinformatics* 23, 2 (2007), 232–239.
- [27] Yuanyuan Tian and Jignesh M Patel. 2008. TALE: A tool for approximate large graph matching. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 963–972.
- [28] Julian R Ullmann. 1976. An algorithm for subgraph isomorphism. *Journal of the ACM (JACM)* 23, 1 (1976), 31–42.
- [29] Andrzej Wasylkowski, Andreas Zeller, and Christian Lindig. 2007. Detecting object usage anomalies. In *Proceedings of the Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering (ESEC/FSE)*. 35–44.
- [30] Qi Wen, Yutong Ye, Xiang Lian, and Mingsong Chen. 2025. S3AND: Efficient subgraph similarity search under aggregated neighbor difference semantics. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3708–3720.
- [31] Guo-Wei Wong, Yi-Ting Huang, Ying-Ren Guo, Ming-Chuan Yang, Shou-De Lin, Wang-Chien Lee, and Meng Chang Chen. 2025. Poster: When logs misbehave: Retrieving known APTs from noisy graphs. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 4776–4778.
- [32] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 590–604.
- [33] Xifeng Yan, Philip S Yu, and Jiawei Han. 2005. Substructure similarity search in graph databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 766–777.
- [34] Linglin Yang, Xunbin Su, Lei Zou, Xiangyang Gou, and Yinnian Lin. 2026. CEMR: An effective subgraph matching algorithm with redundant extension elimination. *Proceedings of the VLDB Endowment* 19, 7 (2026), 1572–1585.
- [35] Rongjian Yang, Zhijie Zhang, Weiguo Zheng, and Jeffrey Xu Yu. 2023. Fast continuous subgraph matching over streaming graphs via backtracking reduction. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
- [36] Ye Yuan, Guoren Wang, Jeffery Yu Xu, and Lei Chen. 2015. Efficient distributed subgraph similarity matching. *The VLDB Journal* 24, 3 (2015), 369–394.
- [37] Zhiping Zeng, Anthony KH Tung, Jianyong Wang, Jianhua Feng, and Lizhu Zhou. 2009. Comparing stars: On approximating graph edit distance. *Proceedings of the VLDB Endowment* 2, 1 (2009), 25–36.
- [38] Shijie Zhang, Jiong Yang, and Wei Jin. 2010. SAPPER: Subgraph indexing and approximate matching in large graphs. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 1185–1194.
- [39] Zhijie Zhang, Yujie Lu, Weiguo Zheng, and Xuemin Lin. 2024. A comprehensive survey and experimental study of subgraph matching: Trends, unbiasedness, and interaction. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–29.
- [40] Zhijie Zhang and Weiguo Zheng. 2025. BEE: Towards redundancy reduction via block-separator decomposition for subgraph matching. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–27.
- [41] Gaoping Zhu, Xuemin Lin, Ke Zhu, Wenjie Zhang, and Jeffrey Xu Yu. 2012. TreeSpan: Efficiently computing similarity all-matching. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*. 529–540.
- [42] Gaoping Zhu, Ke Zhu, Wenjie Zhang, Xuemin Lin, and Chuan Xiao. 2012. Efficient subgraph similarity all-matching. In *Proceedings of the International Conference on Database Systems for Advanced Applications (DASFAA)*. 455–469.
- [43] Lei Zou, Jinghui Mo, Lei Chen, M Tamer Özsu, and Dongyan Zhao. 2011. gStore: answering SPARQL queries via subgraph matching. *Proceedings of the VLDB Endowment* 4, 8 (2011), 482–493.

A APPENDIX

A.1 Proof of Theorem 1

PROOF. For every reachable AFE state $S = (M, c, X)$, the following invariant holds: M is injective and label preserving; the query edges preserved inside $\text{dom}(M)$ connect all mapped query vertices; X contains exactly the anchor edges already committed to be missing; and c equals $|X|$ plus the number of missing nonexcluded query edges whose two endpoints have been mapped. The invariant is true for each root state. An inclusion branch maps a frontier vertex to an unused, label-compatible candidate adjacent to its anchor, enforces all incident exclusions, and charges exactly the newly exposed missing edges. An exclusion branch leaves M unchanged, adds one live anchor edge to X , and increases c by one. Hence the invariant is preserved.

For soundness, if a terminal state reports f , the invariant gives injectivity and label preservation. Every nonroot query vertex was introduced through a preserved anchor edge, so Q_f is connected. Since all query edges are exposed at termination, the invariant also gives $c = \delta(f)$. Only states with $c \leq \theta$ are explored; thus $f \in \mathcal{M}_\theta(Q, G)$.

For completeness, fix $f \in \mathcal{M}_\theta(Q, G)$. The deletion-aware filter is safe, so $f(u) \in C(u)$ for every u , and AFE starts the root branch $r \mapsto f(r)$. Call a state f -consistent when f extends M and every edge in X is missing under f . At an f -consistent state, $c \leq \delta(f) \leq \theta$. Consider its exclusion chain. Because the chain stops before either exceeding the remaining budget or excluding all anchor edges that can connect an unmapped component, f must preserve one of the examined anchor edges. Let (u, a) be the first such edge. All earlier examined edges are missing under f , while $f(u)$ is an unused candidate adjacent to $M(a)$, satisfies X , and adds only missing edges of f to the cost. Therefore, its inclusion branch is explored and reaches another f -consistent state with one more mapped vertex. Induction reaches a terminal state reporting f .

For uniqueness, the value $f(r)$ fixes one root branch. In every later exclusion chain, f belongs only to the branch of the first examined anchor edge that it preserves, and within that branch only to the candidate $f(u)$. Thus f follows one root-to-leaf path and is reported exactly once. $\square$

A.2 Proof of Lemma 1

PROOF. For a Cost-Split state $S = (M, W, c, X)$, let $\mathcal{R}(S)$ be the injective concrete mappings obtained by selecting one value from each white candidate set while respecting X . We show that each operation partitions $\mathcal{R}(S)$ without changing the represented mappings or their exact costs.

Fixing a white anchor or a white neighbor branches on its selected candidate. These branches are disjoint and exhaustive, and the cost is unchanged because all edges from that vertex to previously processed vertices have already been accounted for. Excluding a live anchor edge similarly preserves the represented mappings after restricting them to those in which the edge is missing; the added unit is never charged again.

After the anchor is black, consider a preserved-edge expansion. In black mode, each feasible value v of the frontier vertex u defines one branch. The algorithm first charges the missing nonexcluded edges from u to black vertices. For every white neighbor w , it then

partitions $W(w)$ into values adjacent and nonadjacent to v ; the two groups respectively preserve and miss (u, w) , with one additional unit charged only for the latter unless the edge is already in X . These partitions are disjoint and exhaustive, so every successor has one exact cost and every feasible concrete extension preserving the anchor edge appears in exactly one successor. Since u becomes black, no white–white edge is introduced.

In white mode, all current white neighbors of u are fixed first; hence, every processed neighbor of u is black. The feasible candidates of u are then partitioned into the disjoint buckets $W_d(u)$ according to their exact incremental cost d . Each nonempty bucket yields one successor with cost $c+d$. Candidates in a bucket therefore have a uniform exact cost, the selected anchor edge is preserved for all of them, and making u white creates no white–white query edge. A concrete extension uniquely determines both the preceding fixing branches and its bucket, proving disjointness and completeness.

Thus, both modes preserve all well-formedness conditions and produce exactly the feasible concrete extensions of the selected preserved edge. At a terminal state, no query edge has two white endpoints, so every edge contribution is already included in c . Terminal White Enumeration lists every injective tuple from the remaining white sets once, hence exactly $\mathcal{R}(S)$. Finally, every processed nonroot vertex has a preserved anchor edge, and every white candidate remains adjacent to its fixed anchor, so each reported realization has a connected preserved query graph. This proves the lemma. $\square$